Федеральное государственное автономное образовательное учреждение высшего образования "Национальный исследовательский университет "Высшая школа экономики" Московский институт электроники и математики им. А. Н. Тихонова НИУ ВШЭ Департамент прикладной математики

Практическая работа №3

«Изучение процессов в операционной системе Linux»

По дисциплине «Операционные системы»

Группа: БПМ244

Студент: Артемьев Никита

Москва 2026

Цель работы

Изучить основы управления процессами в Linux, научиться запускать и управлять фоновыми процессами, выполнять мониторинг и логирование с помощью sysstat и скриптов, создать службу на systemd и настроить запуск задач по расписанию через cron.

1. Изучение основ управления процессами

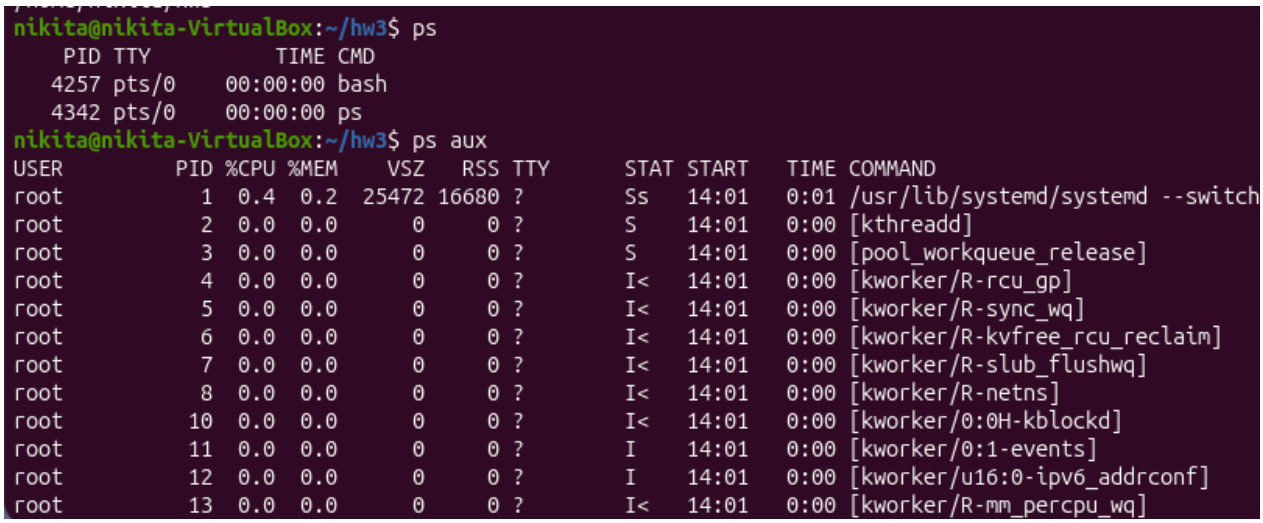
- **ps — делает статический снимок работающих процессов в данный момент.**
- **top — показывает динамический список процессов в реальном времени с автоматическим обновлением.**
- **htop — интерактивная, цветная и более удобная версия top с поддержкой мыши и подсказками.**
- **kill — завершает процесс или отправляет ему специальный сигнал по его номеру (PID).**
- **nice — запускает новую задачу с измененным приоритетом использования процессора.**
- **renice — меняет приоритет процессора для уже запущенного процесса.**

1.1.Просмотр процессов: ps

Команда ps без ключей показывает процессы текущей сессии (оболочка и сама команда ps). Расширенный список получен командой ps aux: видны пользователь, PID, доля CPU и памяти, состояние и командная строка для всех процессов системы.

Команда ps -ef выводит процессы в формате UID/PID/PPID и позволяет проследить иерархию (например, дочерние процессы относительно systemd и kthreadd).

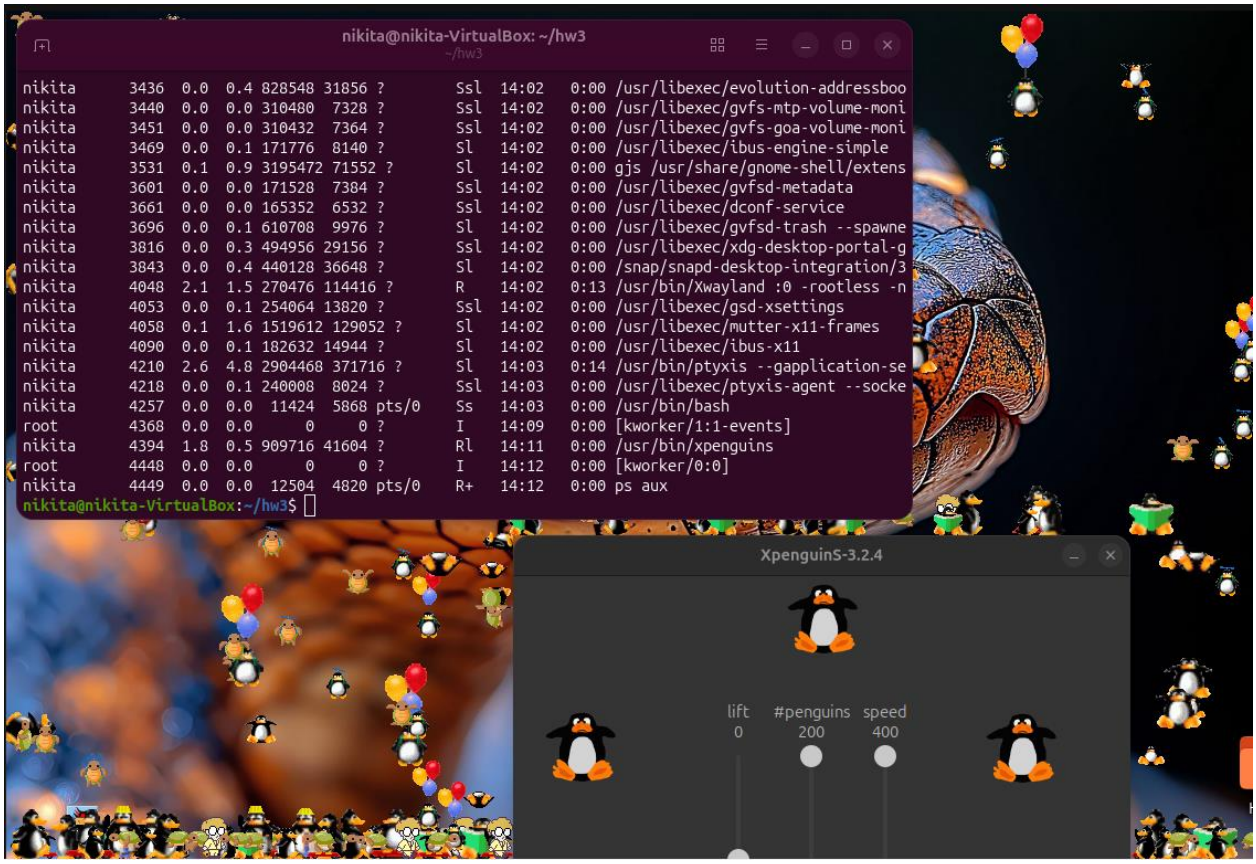
Для отбора полей и сортировки использована команда ps -eo pid,ppid,ni,stat,%cpu,%mem,cmd --sort=-%cpu | head -n 15 — выводятся 15 процессов с наибольшей загрузкой CPU (поля nice, состояние, память).



```
nikita      3404  0.0  0.1 389556 11292 ?        Ssl  14:02   0:00 /usr/libexec/gvfs-udisks2-volume-
nikita      3420  0.0  0.1 311448  7884 ?        Ssl  14:02   0:00 /usr/libexec/gvfs-gphoto2-volume-
nikita      3427  0.0  0.1 392484  9488 ?        Ssl  14:02   0:00 /usr/libexec/gvfs-afc-volume-moni
nikita      3436  0.0  0.4 828548 31856 ?        Ssl  14:02   0:00 /usr/libexec/evolution-addressboo
nikita      3440  0.0  0.0 310480  7328 ?        Ssl  14:02   0:00 /usr/libexec/gvfs-mtp-volume-moni
nikita      3451  0.0  0.0 310432  7364 ?        Ssl  14:02   0:00 /usr/libexec/gvfs-goa-volume-moni
nikita      3469  0.0  0.1 171776  8136 ?        Sl   14:02   0:00 /usr/libexec/ibus-engine-simple
nikita      3531  0.2  0.9 3195472 71556 ?       Sl   14:02   0:00 gjs /usr/share/gnome-shell/extens
nikita      3601  0.0  0.0 171528  7384 ?        Ssl  14:02   0:00 /usr/libexec/gvfsd-metadata
nikita      3661  0.0  0.0 165352  6532 ?        Ssl  14:02   0:00 /usr/libexec/dconf-service
nikita      3696  0.0  0.1 610708  9976 ?        Sl   14:02   0:00 /usr/libexec/gvfsd-trash --spawne
nikita      3816  0.0  0.3 494956 29156 ?        Ssl  14:02   0:00 /usr/libexec/xdg-desktop-portal-g
nikita      3843  0.1  0.4 440128 36648 ?        Sl   14:02   0:00 /snap/snapd-desktop-integration/3
nikita      4048  2.3  1.2 236748 95460 ?        S    14:02   0:08 /usr/bin/Xwayland :0 -rootless -n
nikita      4053  0.0  0.1 254064 13816 ?        Ssl  14:02   0:00 /usr/libexec/gsd-xsettings
nikita      4058  0.2  1.6 1501764 125764 ?      Sl   14:02   0:00 /usr/libexec/mutter-x11-frames
nikita      4090  0.0  0.1 182632 14944 ?        Sl   14:02   0:00 /usr/libexec/ibus-x11
nikita      4210  0.7  3.0 2770864 233692 ?      Sl   14:03   0:01 /usr/bin/ptyxis --gapplication-se
nikita      4218  0.0  0.1 240008  8024 ?        Ssl  14:03   0:00 /usr/libexec/ptyxis-agent --socke
nikita      4257  0.0  0.0  11424  5868 pts/0    Ss   14:03   0:00 /usr/bin/bash
nikita      4344  100  0.0  12504  4820 pts/0    R+   14:07   0:00 ps aux
```

```
nikita@nikita-VirtualBox:~/hw3$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 14:01 ?        00:00:01 /usr/lib/systemd/systemd --switched-root --syste
root           2       0  0 14:01 ?        00:00:00 [kthreadd]
root           3       2  0 14:01 ?        00:00:00 [pool_workqueue_release]
root           4       2  0 14:01 ?        00:00:00 [kworker/R-rcu_gp]
root           5       2  0 14:01 ?        00:00:00 [kworker/R-sync_wq]
root           6       2  0 14:01 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root           7       2  0 14:01 ?        00:00:00 [kworker/R-slub_flushwq]
root           8       2  0 14:01 ?        00:00:00 [kworker/R-netns]
root          10       2  0 14:01 ?        00:00:00 [kworker/0:0H-kblockd]
root          11       2  0 14:01 ?        00:00:00 [kworker/0:1-events]
root          12       2  0 14:01 ?        00:00:00 [kworker/u16:0-ipv6_addrconf]
root          13       2  0 14:01 ?        00:00:00 [kworker/R-mm_percpu_wq]
root          14       2  0 14:01 ?        00:00:00 [ksoftirqd/0]
root          15       2  0 14:01 ?        00:00:00 [rcu_preempt]
root          16       2  0 14:01 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
root          17       2  0 14:01 ?        00:00:00 [rcu_exp_gp_kthread_worker]
```

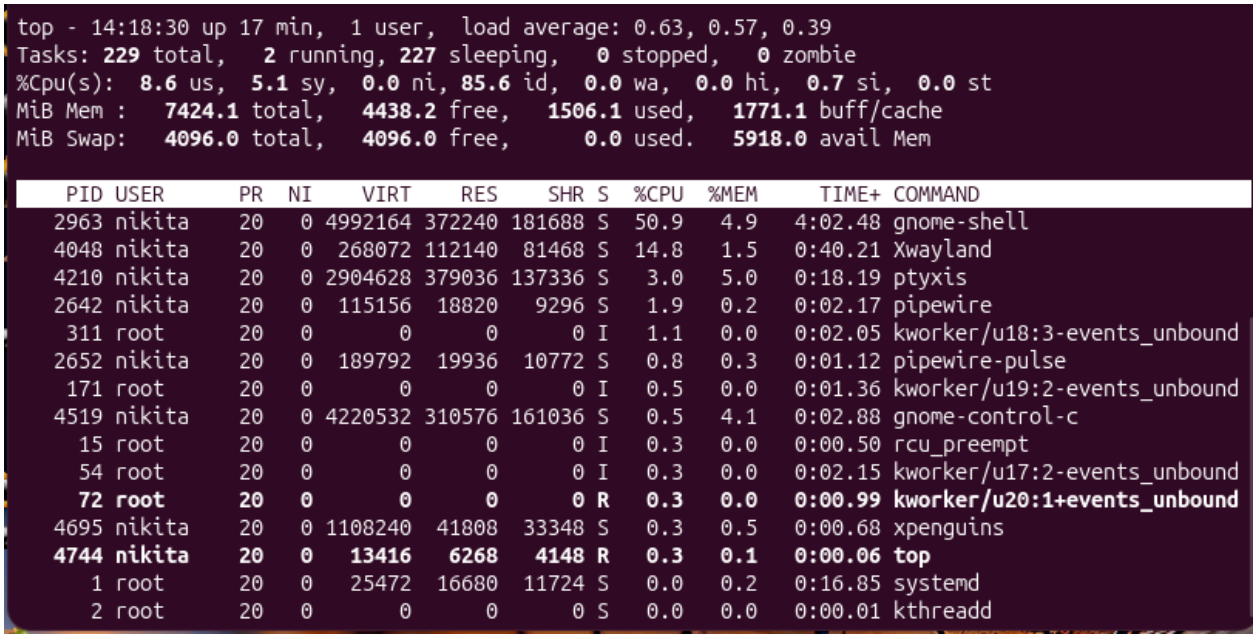
```
nikita@nikita-VirtualBox:~/hw3$ ps -eo pid,ppid,ni,stat,%cpu,%mem,cmd --sort=%cpu | head -n 15
    PID    PPID  NI STAT %CPU %MEM CMD
      3       2   0 S     0.0  0.0 [pool_workqueue_release]
      4       2 -20 I<    0.0  0.0 [kworker/R-rcu_gp]
      5       2 -20 I<    0.0  0.0 [kworker/R-sync_wq]
      6       2 -20 I<    0.0  0.0 [kworker/R-kvfree_rcu_reclaim]
      7       2 -20 I<    0.0  0.0 [kworker/R-slub_flushwq]
      8       2 -20 I<    0.0  0.0 [kworker/R-netns]
     10       2 -20 I<    0.0  0.0 [kworker/0:0H-kblockd]
     12       2   0 I     0.0  0.0 [kworker/u16:0-ipv6_addrconf]
     13       2 -20 I<    0.0  0.0 [kworker/R-mm_percpu_wq]
     16       2   0 S     0.0  0.0 [rcu_exp_par_gp_kthread_worker/0]
     19       2   0 S     0.0  0.0 [kprobe-optimizer]
     20       2   - S     0.0  0.0 [idle_inject/0]
     21       2   0 S     0.0  0.0 [cpuhp/0]
     22       2   0 S     0.0  0.0 [cpuhp/1]
```

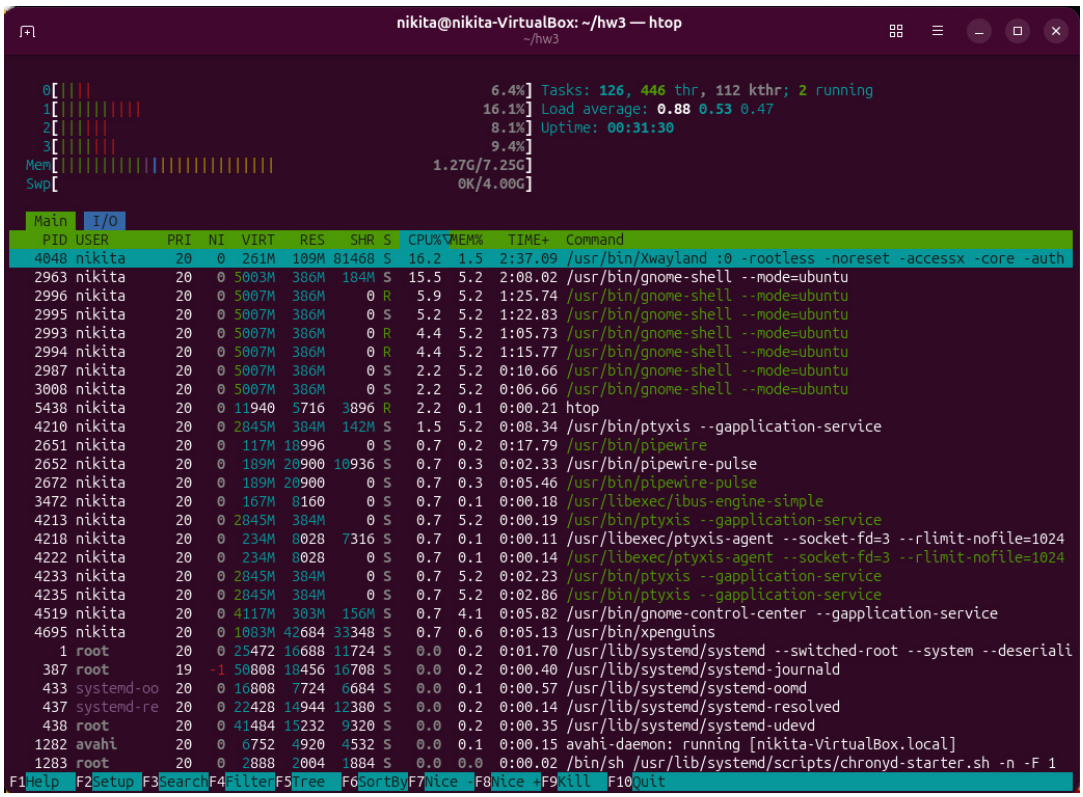


1.2.Мониторинг: top и htop

Интерактивный мониторинг выполнен командой top. В заголовке отображаются load average, число задач, сводка по CPU и памяти; в таблице — процессы с наибольшим %CPU и %MEM (например, gnome-shell, Xwayland).

Утилита htop даёт наглядное представление: загрузка по ядрам, память и swap, список процессов с сортировкой по CPU.





1.3.Приоритеты: nice и renice

```
nikita@nikita-VirtualBox:~/hw3$ nice -n 10 sleep 600 &
[2] 5676
nikita@nikita-VirtualBox:~/hw3$ ps -o pid,ni,cmd -p 5676
    PID  NI CMD
   5676  10 sleep 600
nikita@nikita-VirtualBox:~/hw3$ renice -n 15 -p 5676
5676 (process ID) old priority 10, new priority 15
nikita@nikita-VirtualBox:~/hw3$ ps -o pid,ni,cmd -p 5676
    PID  NI CMD
   5676  15 sleep 600
nikita@nikita-VirtualBox:~/hw3$
```

Процесс sleep 600 запущен в фоне с пониженным приоритетом: nice -n 10 sleep 600 &. Проверка: ps -o pid,ni,cmd -p <PID> — значение **NI = 10**.

Для работающего процесса выполнено renice -n 15 -p <PID>; повторный ps показывает **NI = 15**. Так демонстрируется изменение niceness без перезапуска программы.

1.4 Сигналы kill

Команда kill отправляет процессу сигнал по PID. В п. 2 использованы kill -STOP и kill -CONT; завершение процесса выполняется сигналом по умолчанию (kill PID, SIGTERM).

2. Создание и управление фоновыми процессами

Написана программа на языке C long_task.c: цикл из 100 итераций с паузой 2 с (sleep), вывод номера итерации и PID (getpid()). Сборка: gcc long_task.c -o long_task.

2.1.Запуск в фоне

Запуск в фоновом режиме: ./long_task &. Shell выводит номер задачи и PID; программа печатает «Long task started» и итерации.

```
[1]-  Done                    nice -n 10 sleep 600
nikita@nikita-VirtualBox:~/hw3$ touch long_task.c
nikita@nikita-VirtualBox:~/hw3$ gedit long_task.c
```

Open | long_task.c ~/hw3 | Save

```c
#include <stdio.h>
#include <unistd.h>

int main(){
    printf("Long task started. PID = %d\n", getpid());
    fflush(stdout);

    for(int i = 1; i < 101; ++i){
        printf("Iteration %d of 100\n", i);
        fflush(stdout);
        sleep(2);
    }

    printf("Long task finished\n");
    return 0;
}
```

C | Tab Width: 4

```
nikita@nikita-VirtualBox:~/hw3$ gcc long_task.c -o long_task
nikita@nikita-VirtualBox:~/hw3$ ls
long_task  long_task.c
nikita@nikita-VirtualBox:~/hw3$ ./long_task &
[1] 6414
nikita@nikita-VirtualBox:~/hw3$ Long task started. PID = 6414
Iteration 1 of 100
Iteration 2 of 100
Iteration 3 of 100
Iteration 4 of 100
Iteration 5 of 100
Iteration 6 of 100
Iteration 7 of 100
Iteration 8 of 100
Iteration 9 of 100
Iteration 10 of 100
Iteration 11 of 100
Iteration 12 of 100
```

2.2. jobs, fg, bg

Вывод перенаправлен в файл: ./long_task > long_task.log 2>&1 &. Команда jobs / jobs -l показывает фоновую задачу и PID.

Управление job control: fg %1 — на передний план; **Ctrl+Z** — останов; bg %1 — снова в фоне в состоянии Running.

```
nikita@nikita-VirtualBox:~/hw3$ ls
long_task  long_task.c
nikita@nikita-VirtualBox:~/hw3$ ./long_task > long_task.log 2>&1 &
[1] 6734
nikita@nikita-VirtualBox:~/hw3$ jobs
Files  Running                 ./long_task > long_task.log 2>&1 &
nikita@nikita-VirtualBox:~/hw3$ jobs -l
[1]+  6734 Running                 ./long_task > long_task.log 2>&1 &
nikita@nikita-VirtualBox:~/hw3$
```

```
nikita@nikita-VirtualBox:~/hw3$ jobs -l
[1]+  6734 Stopped (signal)           ./long_task > long_task.log 2>&1
nikita@nikita-VirtualBox:~/hw3$ fg %1
./long_task > long_task.log 2>&1
^Z
[1]+  Stopped                 ./long_task > long_task.log 2>&1
nikita@nikita-VirtualBox:~/hw3$ bg %1
[1]+ ./long_task > long_task.log 2>&1 &
nikita@nikita-VirtualBox:~/hw3$  jobs -l
[1]+  6734 Running                 ./long_task > long_task.log 2>&1 &
```

2.3. kill -STOP и kill -CONT

Остановка без завершения: kill -STOP <PID>. В ps -o pid,stat,cmd состояние **T** (stopped). Возобновление: kill -CONT <PID>, состояние **S** (sleeping/runnable).

```
nikita@nikita-VirtualBox:~/hw3$ kill -STOP 6734

[1]+  Stopped                 ./long_task > long_task.log 2>&1
nikita@nikita-VirtualBox:~/hw3$ ps -o pid,stat,cmd -p 6734
    PID STAT CMD
   6734 T    ./long_task
nikita@nikita-VirtualBox:~/hw3$ kill -COUNT 6734
bash: kill: COUNT: invalid signal specification
nikita@nikita-VirtualBox:~/hw3$ ps -o pid,stat,cmd -p 6734
    PID STAT CMD
   6734 T    ./long_task
nikita@nikita-VirtualBox:~/hw3$ kill -COUNT 6734
bash: kill: COUNT: invalid signal specification
nikita@nikita-VirtualBox:~/hw3$ kill -CONT 6734
nikita@nikita-VirtualBox:~/hw3$ ps -o pid,stat,cmd -p 6734
    PID STAT CMD
   6734 S    ./long_task
```

```
nikita@nikita-VirtualBox:~/hw3$ kill 6734
bash: kill: (6734) - No such process
[1]+  Done                    ./long_task > long_task.log 2>&1
nikita@nikita-VirtualBox:~/hw3$ jobs
nikita@nikita-VirtualBox:~/hw3$ ps -p 6734
    PID TTY          TIME CMD
nikita@nikita-VirtualBox:~/hw3$
```

Сигнал SIGSTOP (команда kill -STOP) переводит процесс в состояние T (остановлен). Сигнал SIGCONT (команда kill -CONT) возобновляет его выполнение. Состояние проверялось командой ps -o pid,stat,cmd -p PID

3.Мониторинг и логирование процессов

Мониторинг ресурсов выполнен пакетом **sysstat** (утилиты sar, iostat, mpstat, pidstat). Они показывают загрузку CPU, память, диск и активность отдельных процессов во времени. Для **логирования** списка процессов использован bash-скрипт log_processes.sh (в задании допускается вместо pslog), результаты пишутся в processes.log; периодический запуск настроен через **cron**.

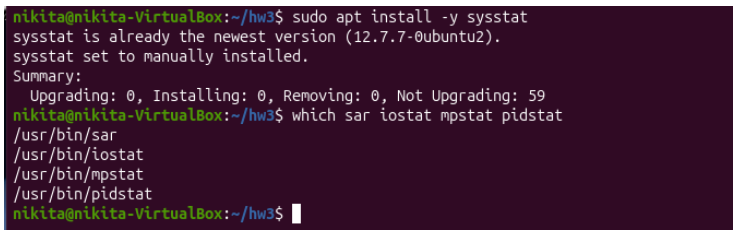
3.1. Установка и настройка sysstat

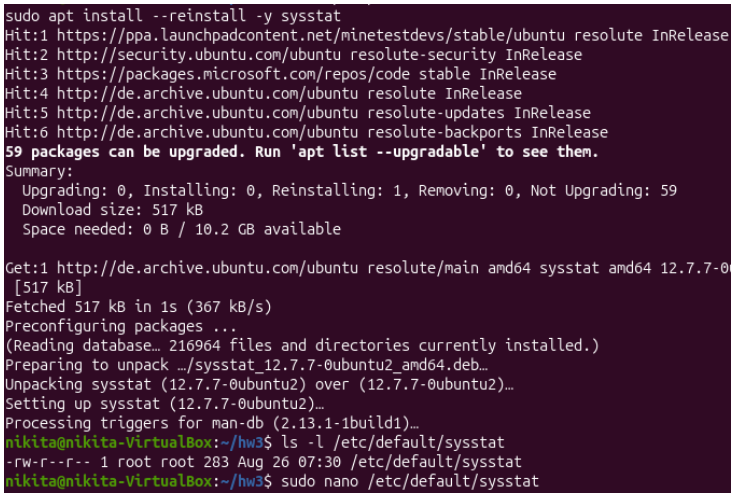
Пакет установлен: sudo apt install sysstat. Наличие утилит проверено: which sar iostat mpstat pidstat.

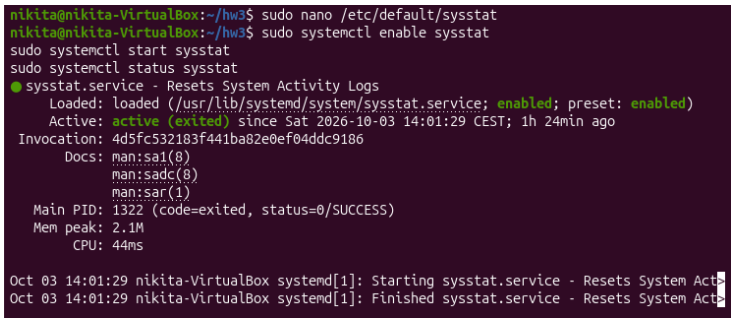
Сбор статистики в фоне управляется службой **sysstat**. В файле /etc/default/sysstat параметр **ENABLED** установлен в **true** (иначе sar может показывать только текущий момент, без истории в /var/log/sysstat/).

Служба включена и запущена:

- sudo systemctl enable sysstat
- sudo systemctl start sysstat
- sudo systemctl status sysstat — состояние **active** (для sysstat часто **active (exited)** — это нормально: unit отработал и запустил сборщики).







3.2.Сбор статистики CPU и памяти (sar)

Команда **sar -u 1 5**: снимки загрузки **CPU** каждые **1** секунду, всего **5** раз. В таблице: %user (пользовательский код), %system (ядро), %iowait (ожидание диска), %idle (простой). Строка **Average** — среднее за интервал.

Команда **sar -r 1 5**: использование **памяти** — свободная/занятая RAM (kB), %memused, кэш, commit и т.д. Так можно оценить, растёт ли потребление памяти за короткий период.

```
nikita@nikita-VirtualBox:~/hw3$ sar -u 1 5
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

03:26:21 PM     CPU     %user     %nice   %system   %iowait    %steal     %idle
03:26:22 PM     all      8.76      0.00      4.24      0.00      0.00     87.01
03:26:23 PM     all      7.34      0.00      3.39      0.00      0.00     89.27
03:26:24 PM     all      6.57      0.00      4.29      0.00      0.00     89.14
03:26:25 PM     all      6.18      0.00      3.09      0.00      0.00     90.73
03:26:26 PM     all      4.52      0.00      3.39      0.00      0.00     92.09
Average:        all      6.67      0.00      3.68      0.00      0.00     89.65
nikita@nikita-VirtualBox:~/hw3$ sar -r 1 5
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

03:26:32 PM kbmemfree   kbavail kbmemused  %memused kbbuffers  kbcached  kbcommit   %commit
  kbactive   kbinact   kbdirty
03:26:33 PM   2219820   5312604   1914576     25.18     62616   3174408   7965848     67.53
    3525236   1438644       656
03:26:34 PM   2223768   5316552   1910628     25.13     62616   3174408   7965860     67.53
    3525236   1438644       656
03:26:35 PM   2219788   5312572   1914600     25.18     62624   3174424   7965924     67.53
    3525220   1438644       672
03:26:36 PM   2219940   5312728   1914404     25.18     62624   3174448   7965912     67.53
    3525204   1438644       684
03:26:37 PM   2220192   5312996   1914152     25.18     62624   3174448   7965796     67.53
    3525188   1438660       684
Average:      2220702   5313490   1913672     25.17     62621   3174427   7965868     67.53
    3525217   1438647       670
```

3.3. Диск и расширенная статистика (iostat)

iostat — нагрузка на **блочные устройства** (диски). Без параметров — сводка с момента загрузки: tps, скорость чтения/записи (kB/s), устройство **sda** — основной диск ВМ.

iostat -x 1 5: расширенные поля каждую 1 с, 5 раз — **%util** (занятость диска), **await** (среднее время I/O), **r/s**, **w/s**. Высокий %util и большой await часто означают узкое место по диску.

```
nikita@nikita-VirtualBox:~/hw3$ iostat
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           4.85    0.03    3.70    0.07    0.00   91.36

Device             tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd
loop0             0.00         0.00         0.00         0.00         17          0          0
loop1             0.01         0.03         0.00         0.00        147          0          0
loop10            0.01         0.02         0.00         0.00        123          0          0
loop11            0.59         2.67         0.00         0.00      13942          0          0
loop12            0.01         0.02         0.00         0.00        125          0          0
loop13            0.35        16.01         0.00         0.00      83708          0          0
loop14            0.01         0.07         0.00         0.00        352          0          0
loop15            0.02         0.34         0.00         0.00       1791          0          0
loop16            0.01         0.07         0.00         0.00        385          0          0
loop17            0.12         4.55         0.00         0.00      23780          0          0
loop18            0.01         0.07         0.00         0.00        374          0          0
loop19            0.01         0.17         0.00         0.00        907          0          0
loop2             0.01         0.03         0.00         0.00        149          0          0
loop20            0.01         0.14         0.00         0.00        729          0          0
loop21            0.00         0.00         0.00         0.00         14          0          0
loop3             0.01         0.13         0.00         0.00        694          0          0
loop4             0.09         1.16         0.00         0.00       6083          0          0
loop5             0.01         0.08         0.00         0.00        410          0          0
loop6             0.45        23.80         0.00         0.00     124437          0          0
loop7             0.01         0.07         0.00         0.00        352          0          0
loop8             0.36         8.66         0.00         0.00      45291          0          0
loop9             0.02         0.56         0.00         0.00       2920          0          0
sda               8.19       461.98       148.58         0.00    2415241     776748          0
```

```
nikita@nikita-VirtualBox:~/hw3$ iostat -x 1 5
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           4.85    0.03    3.70    0.06    0.00   91.35

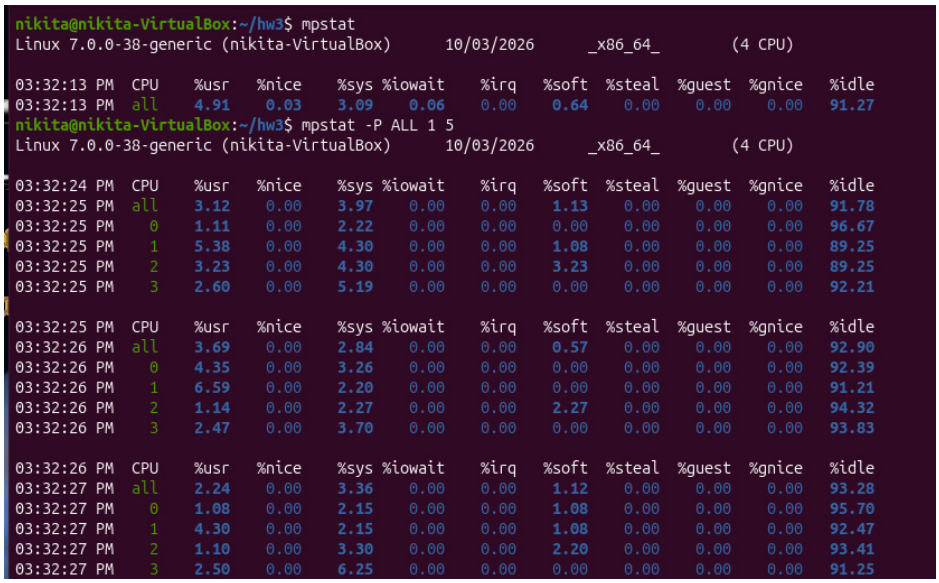
Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s
  %drqm d_await dareq-sz     f/s f_await  aqu-sz  %util
loop0            0.00      0.00     0.00   0.00    0.00     0.00    1.21      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
   0.00    0.00     0.00    0.00    0.00    0.00    0.00
loop1            0.01      0.03     0.00   0.00    0.69     3.27    0.00      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
   0.00    0.00     0.00    0.00    0.00    0.00    0.00
loop10           0.01      0.02     0.00   0.00    1.43     3.32    0.00      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
   0.00    0.00     0.00    0.00    0.00    0.00    0.00
loop11           0.59      2.66     0.00   0.00    0.06     4.49    0.00      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
   0.00    0.00     0.00    0.00    0.00    0.00    0.00
loop12           0.01      0.02     0.00   0.00    1.47     3.29    0.00      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
   0.00    0.00     0.00    0.00    0.00    0.00    0.00
loop13           0.35     15.98     0.00   0.00    0.22    45.69    0.00      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
   0.00    0.00     0.00    0.00    0.00    0.00    0.01
loop14           0.01      0.07     0.00   0.00    0.68     8.00    0.00      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
   0.00    0.00     0.00    0.00    0.00    0.00    0.00
loop15           0.02      0.34     0.00   0.00    0.65    22.11    0.00      0.00     0.00   0.00    0.00     0.00    0.00      0.00     0.00
```

3.4. CPU по ядрам и по процессам (mpstat, pidstat)

mpstat -P ALL 1 5 — загрузка **каждого логического CPU** (0, 1, 2, 3) и строка **all** (среднее). Нужно, чтобы увидеть, упирается ли задача в одно ядро или нагрузка распределена.

pidstat 1 5 — **процессы**, потребляющие CPU: PID, UID, %usr, %system, %CPU, команда. Видно, например, gnome-shell, Xwayland, браузер.

pidstat -r 1 5 — память процессов: minor/major **page faults**, **RSS** (физическая память), **%MEM**. Major faults — обращения к диску за страницами (дороже для производительности).



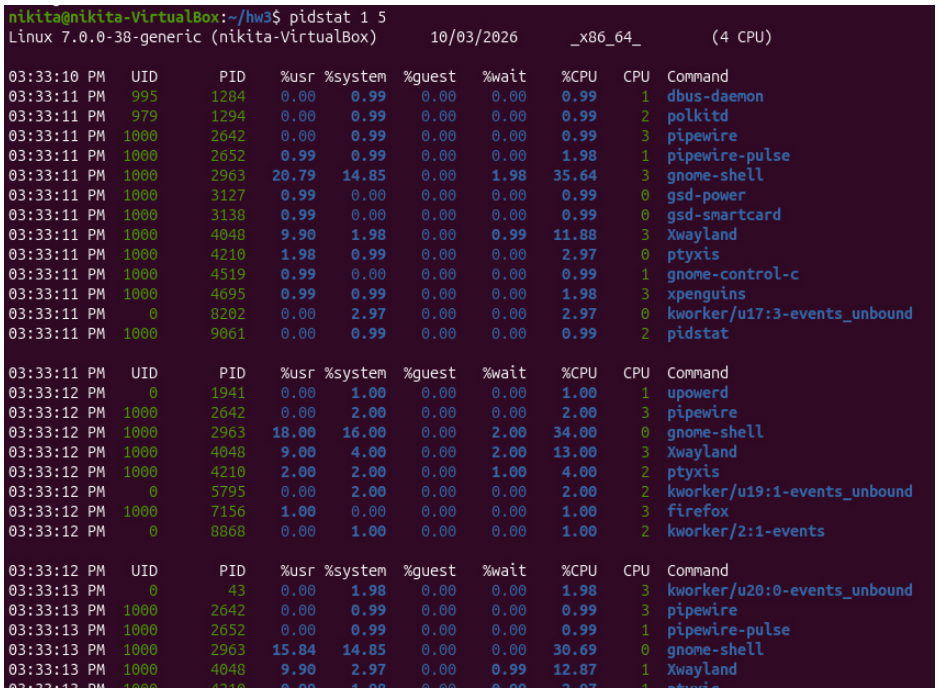
```
nikita@nikita-VirtualBox:~/hw3$ mpstat
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

03:32:13 PM  CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
03:32:13 PM  all    4.91    0.03    3.09    0.06    0.00    0.64    0.00    0.00    0.00   91.27
nikita@nikita-VirtualBox:~/hw3$ mpstat -P ALL 1 5
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

03:32:24 PM  CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
03:32:25 PM  all    3.12    0.00    3.97    0.00    0.00    1.13    0.00    0.00    0.00   91.78
03:32:25 PM    0    1.11    0.00    2.22    0.00    0.00    0.00    0.00    0.00    0.00   96.67
03:32:25 PM    1    5.38    0.00    4.30    0.00    0.00    1.08    0.00    0.00    0.00   89.25
03:32:25 PM    2    3.23    0.00    4.30    0.00    0.00    3.23    0.00    0.00    0.00   89.25
03:32:25 PM    3    2.60    0.00    5.19    0.00    0.00    0.00    0.00    0.00    0.00   92.21

03:32:25 PM  CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
03:32:26 PM  all    3.69    0.00    2.84    0.00    0.00    0.57    0.00    0.00    0.00   92.90
03:32:26 PM    0    4.35    0.00    3.26    0.00    0.00    0.00    0.00    0.00    0.00   92.39
03:32:26 PM    1    6.59    0.00    2.20    0.00    0.00    0.00    0.00    0.00    0.00   91.21
03:32:26 PM    2    1.14    0.00    2.27    0.00    0.00    2.27    0.00    0.00    0.00   94.32
03:32:26 PM    3    2.47    0.00    3.70    0.00    0.00    0.00    0.00    0.00    0.00   93.83

03:32:26 PM  CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
03:32:27 PM  all    2.24    0.00    3.36    0.00    0.00    1.12    0.00    0.00    0.00   93.28
03:32:27 PM    0    1.08    0.00    2.15    0.00    0.00    1.08    0.00    0.00    0.00   95.70
03:32:27 PM    1    4.30    0.00    2.15    0.00    0.00    1.08    0.00    0.00    0.00   92.47
03:32:27 PM    2    1.10    0.00    3.30    0.00    0.00    2.20    0.00    0.00    0.00   93.41
03:32:27 PM    3    2.50    0.00    6.25    0.00    0.00    0.00    0.00    0.00    0.00   91.25
```



```
nikita@nikita-VirtualBox:~/hw3$ pidstat 1 5
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

03:33:10 PM   UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
03:33:11 PM   995      1284    0.00    0.99    0.00    0.00    0.99     1  dbus-daemon
03:33:11 PM   979      1294    0.00    0.99    0.00    0.00    0.99     2  polkitd
03:33:11 PM  1000      2642    0.00    0.99    0.00    0.00    0.99     3  pipewire
03:33:11 PM  1000      2652    0.99    0.99    0.00    0.00    1.98     1  pipewire-pulse
03:33:11 PM  1000      2963   20.79   14.85    0.00    1.98   35.64     3  gnome-shell
03:33:11 PM  1000      3127    0.99    0.00    0.00    0.00    0.99     0  gsd-power
03:33:11 PM  1000      3138    0.99    0.00    0.00    0.00    0.99     0  gsd-smartcard
03:33:11 PM  1000      4048    9.90    1.98    0.00    0.99   11.88     3  Xwayland
03:33:11 PM  1000      4210    1.98    0.99    0.00    0.00    2.97     0  ptyxis
03:33:11 PM  1000      4519    0.99    0.00    0.00    0.00    0.99     1  gnome-control-c
03:33:11 PM  1000      4695    0.99    0.99    0.00    0.00    1.98     3  xpenguins
03:33:11 PM     0      8202    0.00    2.97    0.00    0.00    2.97     0  kworker/u17:3-events_unbound
03:33:11 PM  1000      9061    0.00    0.99    0.00    0.00    0.99     2  pidstat

03:33:11 PM   UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
03:33:12 PM     0      1941    0.00    1.00    0.00    0.00    1.00     1  upowerd
03:33:12 PM  1000      2642    0.00    2.00    0.00    0.00    2.00     3  pipewire
03:33:12 PM  1000      2963   18.00   16.00    0.00    2.00   34.00     0  gnome-shell
03:33:12 PM  1000      4048    9.00    4.00    0.00    2.00   13.00     3  Xwayland
03:33:12 PM  1000      4210    2.00    2.00    0.00    1.00    4.00     2  ptyxis
03:33:12 PM     0      5795    0.00    2.00    0.00    0.00    2.00     2  kworker/u19:1-events_unbound
03:33:12 PM  1000      7156    1.00    0.00    0.00    0.00    1.00     3  firefox
03:33:12 PM     0      8868    0.00    1.00    0.00    0.00    1.00     2  kworker/2:1-events

03:33:12 PM   UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
03:33:13 PM     0        43    0.00    1.98    0.00    0.00    1.98     3  kworker/u20:0-events_unbound
03:33:13 PM  1000      2642    0.00    0.99    0.00    0.00    0.99     3  pipewire
03:33:13 PM  1000      2652    0.00    0.99    0.00    0.00    0.99     1  pipewire-pulse
03:33:13 PM  1000      2963   15.84   14.85    0.00    0.00   30.69     0  gnome-shell
03:33:13 PM  1000      4048    9.90    2.97    0.00    0.99   12.87     1  Xwayland
```

```
nikita@nikita-VirtualBox:~/hw3$ pidstat -r 1 5
Linux 7.0.0-38-generic (nikita-VirtualBox)      10/03/2026      _x86_64_        (4 CPU)

03:33:23 PM   UID       PID  minflt/s  majflt/s     VSZ     RSS   %MEM  Command
03:33:24 PM     0         1    475.25      0.00   26336   17284   0.23  systemd
03:33:24 PM     0       387     60.40      0.00   50948   18032   0.24  systemd-journal
03:33:24 PM  1000      2963  16120.79     63.37 5132844  438268   5.76  gnome-shell
03:33:24 PM  1000      9069      0.99      0.00   10544    4660   0.06  pidstat

03:33:24 PM   UID       PID  minflt/s  majflt/s     VSZ     RSS   %MEM  Command
03:33:25 PM  1000      2963  16901.00     90.00 5132860  438268   5.76  gnome-shell
03:33:25 PM  1000      4210      1.00      0.00 2921648  407172   5.36  ptyxis
03:33:25 PM  1000      9069      2.00      0.00   10544    4660   0.06  pidstat

03:33:25 PM   UID       PID  minflt/s  majflt/s     VSZ     RSS   %MEM  Command
03:33:26 PM  1000      2963  15696.00     72.00 5137020  438268   5.76  gnome-shell

03:33:26 PM   UID       PID  minflt/s  majflt/s     VSZ     RSS   %MEM  Command
03:33:27 PM  1000      2963  14465.00     55.00 5137020  438268   5.76  gnome-shell
03:33:27 PM  1000      5965      2.00      0.00   46400   33604   0.44  python3

03:33:27 PM   UID       PID  minflt/s  majflt/s     VSZ     RSS   %MEM  Command
03:33:28 PM  1000      2963  16170.00     76.00 5137020  438268   5.76  gnome-shell

Average:      UID       PID  minflt/s  majflt/s     VSZ     RSS   %MEM  Command
Average:        0         1     95.81      0.00   26336   17284   0.23  systemd
Average:        0       387     12.18      0.00   50948   18032   0.24  systemd-journal
Average:     1000      2963  15871.06     71.26 5135353  438268   5.76  gnome-shell
Average:     1000      4210      0.20      0.00 2921648  407172   5.36  ptyxis
Average:     1000      5965      0.40      0.00   46400   33604   0.44  python3
Average:     1000      9069      0.60      0.00   10544    4660   0.06  pidstat
```

3.5. Логирование процессов (bash-скрипт)

Утилита **pslog** в репозитории не использовалась; реализовано логирование скриптом **log_processes.sh**:

- в лог дописывается метка времени;
- выполняется ps -eo pid,ppid,user,ni,stat,%cpu,%mem,cmd --sort=-%cpu | head -n 10 — **10 процессов с наибольшим %CPU**;
- файл: $HOME/hw3/processes.log.

Скрипт сделан исполняемым (chmod +x) и запускался вручную (./log_processes.sh), содержимое проверено через cat processes.log.

```
nikita@nikita-VirtualBox:~/hw3$ nano log_processes.sh
nikita@nikita-VirtualBox:~/hw3$ cat log_processes.sh
#!/bin/bash

LOG_FILE="$HOME/hw3/processes.log"

echo "===== $(date '+%Y-%m-%d %H:%M:%S') =====" >> "$LOG_FILE"

ps -eo pid,ppid,user,ni,stat,%cpu,%mem,cmd --sort=-%cpu | head -n 10 >> "$LOG_FILE"

echo "" >> "$LOG_FILE"
nikita@nikita-VirtualBox:~/hw3$ chmod +x log_processes.sh
nikita@nikita-VirtualBox:~/hw3$ ./log_processes.sh
```

```
nikita@nikita-VirtualBox:~/hw3$ cat processes.log
===== 2026-10-03 15:51:37 =====
    PID    PPID USER      NI STAT %CPU %MEM CMD
   2963    2603 nikita     0 Ssl  33.7  5.7 /usr/bin/gnome-shell --mode=ubuntu
   4048    2963 nikita     0 S    12.4  1.4 /usr/bin/Xwayland :0 -rootless -noreset -accessx -core -auth /
run/user/1000/.mutter-Xwaylandauth.OSHVW3 -listenfd 4 -listenfd 5 -displayfd 6 -initfd 7 -byteswappedclien
ts -enable-ei-portal
   7156    2963 nikita     0 Sl    1.9  6.7 /snap/firefox/8763/usr/lib/firefox/firefox
   2642    2603 nikita     0 Ssl   1.5  0.2 /usr/bin/pipewire
   4210    2603 nikita     0 Sl    0.9  5.3 /usr/bin/ptyxis --gapplication-service
   2652    2603 nikita     0 SLsl  0.7  0.2 /usr/bin/pipewire-pulse
   7917    7336 nikita     0 Sl    0.5  2.9 /snap/firefox/8763/usr/lib/firefox/firefox -contentproc -isFor
Browser -prefsHandle 0:51401 -prefMapHandle 1:293639 -jsInitHandle 2:161776 -parentBuildID 20260814105733
-sandboxReporter 3 -chrootClient 4 -ipcHandle 5 -initialChannelId {7b31b0bf-3d09-4a2b-9ba1-d7141f123f95} -
parentPid 7156 -crashHelperPid 7240 -crashHelper 6 -crashReporter 7 -greomni /snap/firefox/8763/usr/lib/fi
refox/omni.ja -appomni /snap/firefox/8763/usr/lib/firefox/browser/omni.ja -appDir /snap/firefox/8763/usr/l
ib/firefox/browser 7 tab
   4695    2963 nikita     0 Sl    0.5  0.5 /usr/bin/xpenguins
   4519    2603 nikita     0 Sl    0.5  4.0 /usr/bin/gnome-control-center --gapplication-service
```

3.6. Автоматизация логирования (cron)

Чтобы лог собирался без ручного запуска, в **crontab** пользователя добавлена строка: */5****/home/nikita/hw3/log_processes.sh - запуск **каждые 5 минут**. Проверка: crontab -l.

```
nikita@nikita-VirtualBox:~/hw3$ crontab -e
no crontab for nikita - using an empty one
Select an editor.  To change later, run select-editor again.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.tiny
  3. /usr/bin/code
  4. /bin/ed

Choose 1-4 [1]: 1
crontab: installing new crontab
nikita@nikita-VirtualBox:~/hw3$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/5 * * * * /home/nikita/hw3/log_processes.sh
```

4.Изучение демонов и служб

В Linux долгоживущие фоновые задачи (демоны) часто оформляют как **службы systemd**. Systemd загружает **unit-файлы**, отслеживает состояние процесса, может перезапускать его при падении и включать автозапуск при загрузке системы. Для обзора системы использована команда systemctl; для практики создана собственная служба **mydaemon.service**, запускающая bash-скрипт mydaemon.sh.

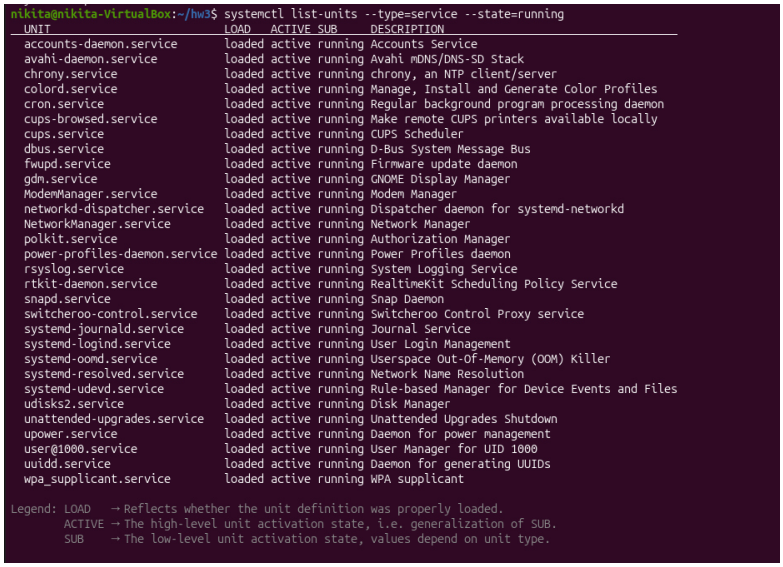
4.1. Активные службы в системе

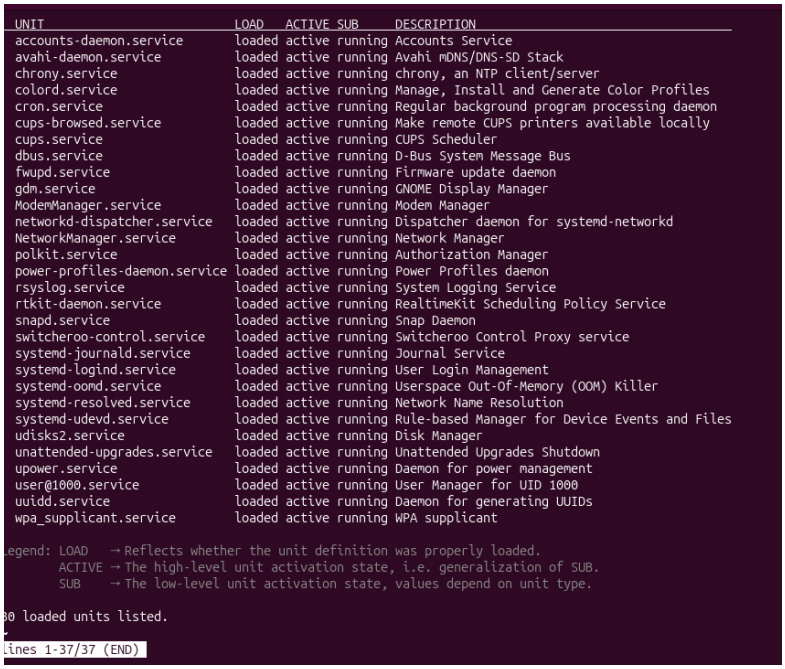
Список работающих служб получен командой: systemctl list-units --type=service --state=running

В выводе для каждой unit-записи указаны:

- **UNIT** — имя службы (например, cron.service, NetworkManager.service, systemd-journald.service);
- **LOAD** — unit загружен из конфигурации;
- **ACTIVE / SUB** — служба **active (running)**;
- **DESCRIPTION** — краткое назначение.

Так видно, какие системные демоны уже работают после загрузки (сеть, журнал, cron, дисплейный менеджер и т.д.).





4.2. Что такое демон в этой работе

Демон — процесс, который работает в фоне, не привязан к интерактивному терминалу и выполняет периодическую или постоянную работу. В учебной работе «демон» реализован **скриптом** с бесконечным циклом: раз в 5 секунд в лог пишется строка с текущим временем. Управление жизненным циклом (старт, автозапуск, перезапуск) делегировано **systemd**, а не ручному nohup ... &.

4.3. Скрипт mydaemon.sh

Создан исполняемый скрипт **~/hw3/mydaemon.sh**:

- переменная LOG_FILE="$HOME/hw3/daemon.log";
- цикл while true: запись строки Дата/время — Daemon is running в лог (>>), затем sleep 5.

Права: chmod +x mydaemon.sh. Так имитируется фоновая служба с периодической активностью; результат проверяется по файлу **daemon.log**.

```
nikita@nikita-VirtualBox:~/hw3$ nano mydaemon.sh
nikita@nikita-VirtualBox:~/hw3$ cat mydaemon.sh
#!/bin/bash

LOG_FILE="$HOME/hw3/daemon.log"

while true; do
    echo "$(date '+%Y-%m-%d %H:%M:%S') Daemon is running" >> "$LOG_FILE"
    sleep 5
done
nikita@nikita-VirtualBox:~/hw3$ chmod +x mydaemon.sh
```

4.4. Unit-файл systemd и запуск службы

Unit-файл **/etc/systemd/system/mydaemon.service**:

Секция	**Назначение**
[Unit]	Description=My Custom Daemon — описание для systemctl status.
[Service]	ExecStart=/home/nikita/hw3/mydaemon.sh — команда запуска; Restart=always — перезапуск при завершении/ошибке; User=nikita — процесс от пользователя nikita (лог в домашнем каталоге).
[Install]	WantedBy=multi-user.target — включение службы в обычный многопользовательский режим загрузки.

После создания unit-файла:

1. sudo systemctl daemon-reload — перечитать конфигурацию systemd;

2. sudo systemctl start mydaemon.service — запустить службу;
3. sudo systemctl status mydaemon.service — убедиться в состоянии **active (running)**, увидеть **Main PID** и процесс mydaemon.sh.

```
nikita@nikita-VirtualBox:~/hw3$ sudo nano /etc/systemd/system/mydaemon.service
nikita@nikita-VirtualBox:~/hw3$ cat /etc/systemd/system/mydaemon.service
[Unit]
Description=My Custom Daemon

[Service]
ExecStart=/home/nikita/hw3/mydaemon.sh
Restart=always
User=nikita

[Install]
WantedBy=multi-user.target
nikita@nikita-VirtualBox:~/hw3$ sudo systemctl daemon-reload
sudo systemctl start mydaemon.service
sudo systemctl status mydaemon.service
● mydaemon.service - My Custom Daemon
     Loaded: loaded (/etc/systemd/system/mydaemon.service; disabled; preset: enabled)
     Active: active (running) since Sat 2026-10-03 16:08:24 CEST; 25ms ago
 Invocation: d039f00482c045fa9f63d87f301b3f8e
   Main PID: 9943 (mydaemon.sh)
      Tasks: 2 (limit: 6336)
     Memory: 2M (peak: 2.2M)
        CPU: 13ms
     CGroup: /system.slice/mydaemon.service
             ├─9943 /bin/bash /home/nikita/hw3/mydaemon.sh
             └─9948 date "+%Y-%m-%d %H:%M:%S"

Oct 03 16:08:24 nikita-VirtualBox systemd[1]: Started mydaemon.service - My Custom Daemon.
nikita@nikita-VirtualBox:~/hw3$
```

4.5. Проверка работы и автозапуск

Работа скрипта подтверждена файлом **daemon.log**: записи с интервалом около 5 секунд (cat ~/hw3/daemon.log).

Автозапуск при загрузке: sudo systemctl enable mydaemon.service Systemd создаёт симлинк в multi-user.target.wants. После перезагрузки служба должна стартовать автоматически (при сдаче можно не перезагружать ВМ, достаточно скрина enable + running status).

```
nikita@nikita-VirtualBox:~/hw3$ cat ~/hw3/daemon.log
2026-10-03 16:08:24 Daemon is running
2026-10-03 16:08:29 Daemon is running
2026-10-03 16:08:34 Daemon is running
2026-10-03 16:08:39 Daemon is running
2026-10-03 16:08:44 Daemon is running
2026-10-03 16:08:49 Daemon is running
2026-10-03 16:08:54 Daemon is running
2026-10-03 16:08:59 Daemon is running
nikita@nikita-VirtualBox:~/hw3$ sudo systemctl enable mydaemon.service
Created symlink '/etc/systemd/system/multi-user.target.wants/mydaemon.service' → '/etc/systemd/system/mydaemon.service'.
```

4.6. Отличие от «просто & в фоне»

Запуск ./mydaemon.sh & зависит от сессии терминала и не перезапустит скрипт после сбоя. **systemd** даёт единый интерфейс (start/stop/status/enable), политику **Restart=**, логи через **journalctl -u mydaemon.service** и интеграцию с загрузкой системы.

5.Автоматизация задач с помощью скриптов

Планировщик **cron** запускает команды и скрипты по расписанию. Задания пользователя хранятся в **crontab** (crontab -e / crontab -l). В работе настроены два задания: периодическое логирование процессов (связь с п. 3) и ежедневная проверка с **уведомлением об ошибке** через запись в лог и (опционально) отправку в мессенджер.

5.1.Скрипт daily_check.sh

Создан скрипт **~/hw3/daily_check.sh**, который проверяет наличие файла **important.txt** в каталоге ~/hw3. **Логика:**

- если файла **нет** — вызывается функция **notify** с текстом ошибки, скрипт завершается с кодом **1** (ошибка для cron);
- если файл **есть** — в лог пишется сообщение **OK**, код выхода **0**.

Лог уведомлений: **$HOME/hw3/notifications.log** (каждая запись с меткой времени). **Функция notify** реализует «уведомление при ошибке»:

- всегда дописывает строку **ERROR** в notifications.log;
- при заданных **BOT_TOKEN** и **CHAT_ID** может отправить сообщение в **Telegram** через HTTP-запрос к Bot API (curl). На практике для демонстрации достаточно лога; API включается при наличии токена (без публикации секретов в отчёте).

```
nikita@nikita-VirtualBox:~/hw3$ nano daily_check.sh
nikita@nikita-VirtualBox:~/hw3$ cat daily_check.sh
#!/bin/bash

# === Настройки ===
LOG_FILE="$HOME/hw3/notifications.log"
CHECK_FILE="$HOME/hw3/important.txt"

# Функция уведомления
notify() {
    local MESSAGE="$1"
    local TIMESTAMP
    TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

    # Пишем в лог
    echo "[$TIMESTAMP] ERROR: $MESSAGE" >> "$LOG_FILE"
}

# === Основная логика ===
if [ ! -f "$CHECK_FILE" ]; then
    notify "Файл $CHECK_FILE не найден!"
    exit 1
fi

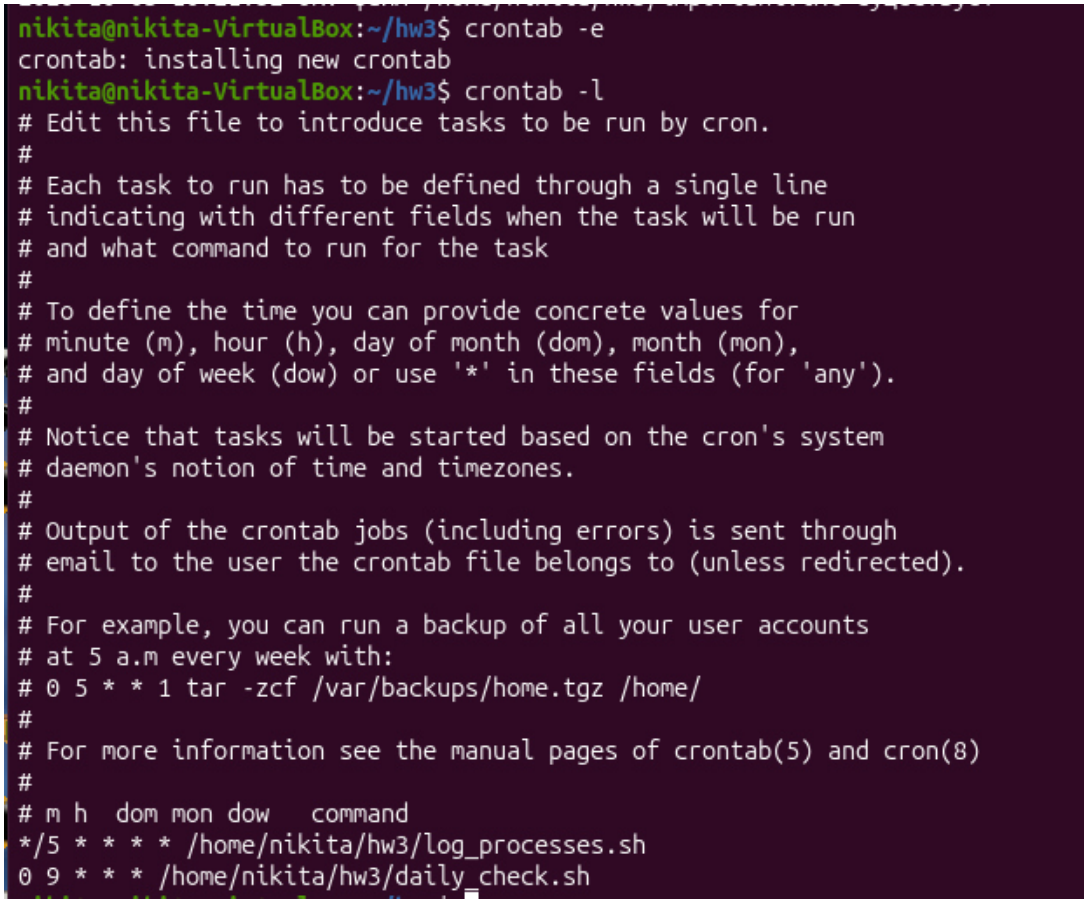
echo "$(date '+%Y-%m-%d %H:%M:%S') OK: файл $CHECK_FILE существует" >> "$LOG_FILE"
exit 0
nikita@nikita-VirtualBox:~/hw3$ chmod +x daily_check.sh
nikita@nikita-VirtualBox:~/hw3$ ./daily_check.sh
nikita@nikita-VirtualBox:~/hw3$ touch important.txt
nikita@nikita-VirtualBox:~/hw3$ ./daily_check.sh
nikita@nikita-VirtualBox:~/hw3$ cat notifications.log
[2026-10-03 16:21:08] ERROR: Файл /home/nikita/hw3/important.txt не найден!
2026-10-03 16:21:52 OK: файл /home/nikita/hw3/important.txt существует
```

5.2. Задания в crontab

Редактирование: crontab -e, просмотр: crontab -l.

Расписание	**Команда**	**Смысл**
*/5 * * * *	/home/nikita/hw3/log_processes.sh	каждые **5 минут** — снимок топ-10 процессов в processes.log (п. 3)
0 9 * * *	/home/nikita/hw3/daily_check.sh	каждый день в **09:00** — проверка important.txt и при необходимости ERROR в notifications.log

Формат cron: **минута час день_месяца месяц день_недели команда**. Звёздочка * — «любое значение»; */5 в поле минут — «каждые 5 минут».



5.3. Поведение cron при ошибках

При ненулевом коде выхода (daily_check.sh → **1**) cron может отправить письмо локальному пользователю **если** настроен MTA (mail). В учебной среме почта часто не настроена, поэтому основной канал уведомления — **notifications.log** и опционально Telegram в **notify()**. Так выполняется требование задания об уведомлении при ошибках без обязательной реальной почты на ВМ.

Выводы

В ходе работы изучены основные средства управления процессами в Linux: просмотр и отбор процессов (ps), интерактивный мониторинг (top, htop), изменение приоритета (nice, renice) и управление сигналами (kill, в том числе STOP/CONT).

Реализована программа на C с длительным выполнением и отработаны механизмы фонового запуска (&), job control (jobs, fg, bg) и остановки/продолжения процесса без его завершения.

Настроен мониторинг с пакетом sysstat (sar, iostat, mpstat, pidstat) и организовано логирование наиболее нагруженных процессов скриптом log_processes.sh с периодическим запуском через cron.

Создана пользовательская служба **mydaemon.service** на базе systemd: автоматический запуск, перезапуск и включение в автозагрузку; работа подтверждена файлом daemon.log.

Настроен cron для регулярных задач; скрипт daily_check.sh демонстрирует обработку ошибочной ситуации (отсутствие файла) с записью уведомления в лог и возможностью оповещения через мессенджер. Полученные навыки применимы для администрирования, диагностики производительности и автоматизации типовых задач в Linux.